

DOCUMENTACIÓN TÉCNICA: PROYECTO DE SISTEMA POS (Marcel's)

Asignatura: Técnicas de Programación – Universidad de Antioquia (UdeA)

1. ARQUITECTURA GENERAL DEL SISTEMA

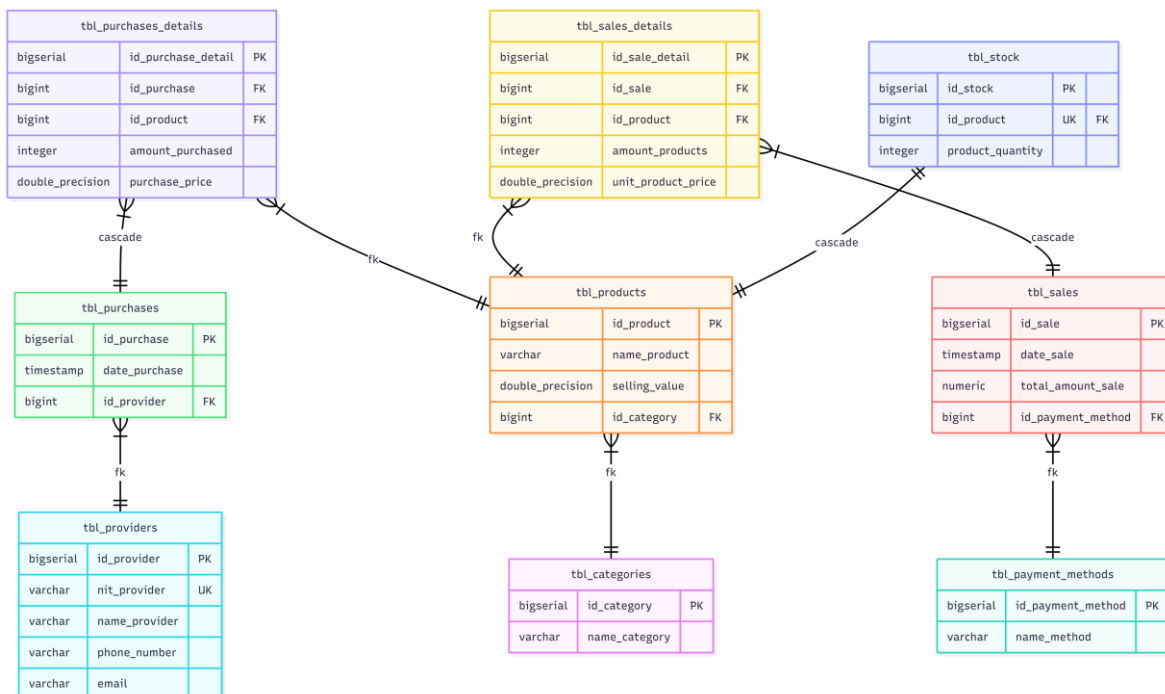
El sistema Marcel's ha sido estructurado bajo un patrón de **Monolito Modular, utilizando también un principio de capas**, diseñado para maximizar la mantenibilidad y el cumplimiento de los principios SOLID. La solución integra componentes que se comunican de manera segura mediante servicios web REST:

- **Capa de Presentación (Frontend):** Desarrollada con Next.js y estilizada con Bootstrap. Es el componente encargado de la visualización de indicadores (KPIs), el flujo de facturación, el control de inventarios y la gestión de tablas dinámicas.
- **Capa de Negocio (Backend):** Implementada en Spring Boot 3.x con Java 21, estructurada bajo el patrón **Controller-Service-Repository**. Esta segmentación garantiza que la lógica de negocio permanezca aislada de las tecnologías de persistencia y transporte de datos.
- **Capa de Datos:** PostgreSQL relacional administrado en el clúster serverless de **Neon.tech**. Se implementaron protocolos de seguridad mediante canales cifrados SSL para asegurar la integridad transaccional (ACID) del sistema.

2. MODELADO CONCEPTUAL Y DISEÑO ESTRUCTURAL

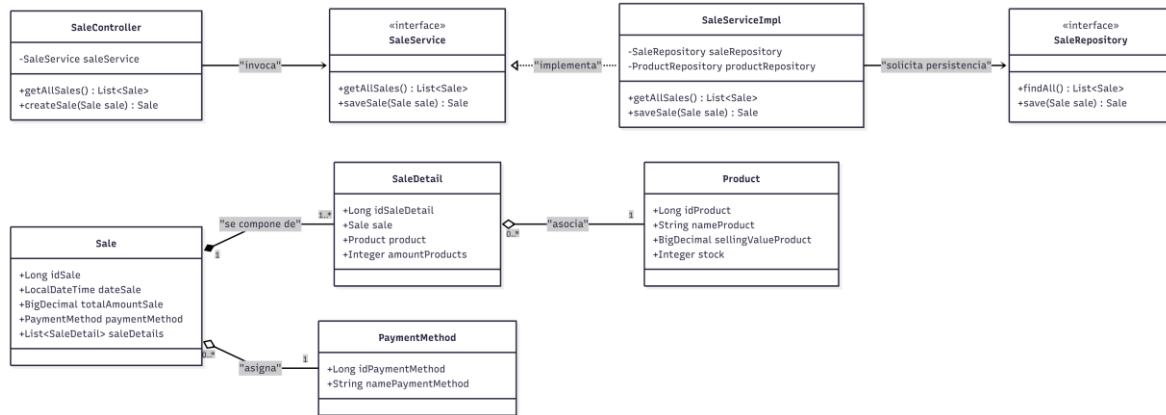
2.1 Diagrama Entidad-Relación (DER)

Este diagrama ilustra la persistencia de los datos, organizado en módulos que permiten la trazabilidad de productos, proveedores, compras y transacciones de venta.



2.2 Diagrama de Clases (Arquitectura del Backend)

El siguiente diagrama detalla la organización de los objetos y las relaciones entre los controladores de la API, los contratos de servicios y los repositorios de datos que gestionan la inyección de dependencias en Spring Boot.



3. ESPECIFICACIÓN DEL MODELO DE DATOS Y PERSISTENCIA (JPA)

La comunicación entre el clúster de Neon.tech y la aplicación Java se realiza mediante JPA (Java Persistence API) y Hibernate. Se utilizó BigDecimal para blindar las operaciones contables contra errores de redondeo numérico, y se emplearon anotaciones de gestión de ciclo de vida (@JsonManagedReference) para evitar errores de serialización.

A continuación, se presenta la implementación de la entidad principal de ventas:

Java

@Entity

@Table(name = "tbl_sales")

@Data

@NoArgsConstructor

@AllArgsConstructor

public class Sale {

 @Id

 @GeneratedValue(strategy = GenerationType.IDENTITY)

 @Column(name = "id_sale")

 private Long idSale;

 @Column(name = "date_sale", nullable = false)

 private LocalDateTime dateSale;

```

@Column(name = "total_amount_sale", nullable = false)
private BigDecimal totalAmountSale;

@ManyToOne(fetch = FetchType.EAGER)
@JoinColumn(name = "id_payment_method", nullable = false)
private PaymentMethod paymentMethod;

@OneToMany(mappedBy = "sale", cascade = CascadeType.ALL, orphanRemoval = true)
@JsonManagedReference
private List<SaleDetail> saleDetails;
}

```

4. LÓGICA DE NEGOCIO Y TRANSACCIONALIDAD

El sistema garantiza la integridad de los datos mediante el uso de transacciones atómicas decoradas con `@Transactional`. Esto asegura que ante cualquier falla en el inventario durante el registro de una venta (por ejemplo, stock insuficiente), el sistema realice un *Rollback* automático, evitando discrepancias financieras.

5. DOCUMENTACIÓN AUTOMATIZADA (SWAGGER / OPENAPI)

El backend incorpora la especificación dinámica de **Springdoc OpenAPI**, eliminando la necesidad de documentación estática y manual. El sistema expone un portal interactivo local en <http://localhost:8080/swagger-ui/index.html>.

Funcionalidad de Sandboxing (Pruebas en vivo):

Esta herramienta permite visualizar contratos de datos (schemas) y ejecutar peticiones HTTP directamente desde la web, facilitando la certificación de las llaves foráneas y los métodos del CRUD sin requerir software externo

6. CONCLUSIONES

1. **Eficiencia en Arquitectura:** La separación de responsabilidades permitió un desarrollo escalable, manteniendo la lógica de persistencia aislada de las interfaces de comunicación con el cliente.
2. **Integridad de Datos:** La implementación de transacciones protegidas y el uso de tipos de datos de alta precisión financiera (`BigDecimal`) aseguran un entorno contable confiable para la gestión de un POS.

3. **Autodocumentación:** La implementación de Swagger facilitó la integración continua entre el frontend y el backend, reduciendo los tiempos de depuración y mejorando la calidad de la entrega final.